

ChromeOS Enterprise Connector

End-to-End Product Documentation

"Halofort" ChromeOS MDM Portal — Feature Breakdown, Google API Mapping,
Chrome Extension Guide, and Development Approach

Prepared: September 2026
Version 1.0 — Internal Engineering Documentation

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	4
2. Technology Stack	4
2.1 Backend.....	4
2.2 Frontend.....	5
2.3 Chrome Extension (“Halo Agent”).....	5
2.4 Summary Table	5
3. Architecture Overview	7
4. End-to-End Feature Breakdown.....	8
4.1 Authentication & Roles	8
4.2 Setup Wizard — Initial Connection	8
4.3 Dashboard — Connector Status.....	9
4.4 Devices	9
4.4.1 Devices Table & Filters	9
4.4.2 Bulk Actions.....	9
4.4.3 Per-Device Actions — Complete List.....	9
4.4.4 Device Detail — Nine Tabs	10
4.4.5 Mobile Devices	11
4.5 Fleet Insights & Reports	11
4.5.1 Fleet Insights	11
4.5.2 Reports	11
4.6 Security	12
4.7 Enrollment.....	12
4.8 Settings.....	12
4.9 Directory & Activity	12
4.10 Theme Templates.....	13
4.11 Real vs. Local-only Actions — Summary	13
5. Google APIs & Metrics Mapping	14
5.1 Google API Surfaces & OAuth Scopes	14
5.2 Device Actions → Exact Google API Method	14
5.3 Fleet Insights Metrics — Data Source	15
5.4 Backend Endpoints — Full Reference	16

5.5 Authentication — Two Separate Systems.....	17
5.6 Mock vs. Real Mode	17
5.7 Data Persistence.....	17
6. Chrome Extension — How It Works and Usage.....	18
6.1 Permissions and Why Each Exists	18
6.2 Device Identification	18
6.3 Location Tracking	18
6.4 Real-Time “Live Check-in” (WebSocket)	18
6.5 Idle / Activity State	19
6.6 Diagnostics and Troubleshooting.....	19
6.7 Manual Device ID Override	19
6.8 Deployment.....	19
6.9 End-User Impact.....	20
7. Development Approach — Manual vs. AI-Assisted	21
7.1 What Actually Happened (AI-Assisted)	21
7.2 Manual-Equivalent Estimate (Industry Baseline).....	21

1. Executive Summary

This document is the complete end-to-end record of the ChromeOS Enterprise Connector (internally themed as “Halofort”) — a self-hosted admin portal that manages a fleet of ChromeOS devices through Google's real, published Workspace/Chrome Enterprise APIs, paired with a companion Chrome extension that reports live location and connectivity signals Google's own APIs cannot provide (a genuine real-time WebSocket connection, rather than a polling guess).

The system is a single Go backend (~6,700 lines) serving a self-contained, single-file React frontend (~7,300 lines, no build step required) and a Manifest V3 Chrome extension (~1,000 lines). It integrates eight distinct Google API surfaces under one service-account identity using domain-wide delegation, and can additionally run in a fully self-contained “mock mode” with no Google credentials at all, for zero-config local development and demos.

At a glance, this documentation covers:

- **Section 4** — every feature end to end: setup, dashboard, devices (including every single device action), fleet insights, reports, security, enrollment, settings, and theming.
- **Section 5** — exactly which Google API and OAuth scope powers every metric and every write action, endpoint by endpoint.
- **Section 6** — how the Chrome extension works internally and how it is deployed/used in practice.
- **Section 7** — an honest, clearly-labeled comparison of the actual AI-assisted build timeline against an industry-standard manual-development estimate for the same scope.
- **Section 2** — the full technology stack.

2. Technology Stack

2.1 Backend

- **Language/runtime** — Go 1.22, standard library net/http server (no web framework — routing is a plain ServeMux with hand-written middleware for CORS and role checks).
- **Google API clients** — official google.golang.org/api v0.183.0 packages: `admin/directory/v1`, `admin/reports/v1`, `alertcenter/v1beta1`, `chromemanagement/v1`, `chromepolicy/v1`.
- **Authentication to Google** — golang.org/x/oauth2 v0.21.0 (`google.JWTConfigFromJSON`), implementing service-account + domain-wide-delegation impersonation — not a user-facing “Sign in with Google” flow.
- **Real-time layer** — github.com/gorilla/websocket v1.5.3, powering the extension's persistent “Live check-in” connection; this is this project's own infrastructure, not a Google API.
- **Persistence** — go.mongodb.org/mongo-driver v2 v2.8.0 (MongoDB), entirely optional: the app runs fully in-memory and still functions if no MongoDB instance is reachable at startup.
- **Password hashing** — golang.org/x/crypto v0.33.0 (`bcrypt`), used for the portal's own login accounts.
- **Portal session auth** — a custom opaque bearer-token scheme (`crypto/rand`, 24-hex-byte tokens, 12-hour TTL) — entirely separate from Google authentication.

2.2 Frontend

- **Delivery model** — a single static HTML file (frontend/index.html, ~7,300 lines) with no bundler, no npm build step, and no server-side templating.
- **UI library** — React 18 and ReactDOM 18, loaded as pre-built UMD development bundles from the unpkg CDN.
- **JSX transpilation** — Babel Standalone 7 (@babel/standalone), loaded from the unpkg CDN and run live in the browser on page load — JSX is authored directly in a `<script type="text/babel">` block.
- **Mapping** — Leaflet 1.9.4 (CDN), powering the live device-location map and trail view.
- **Styling/theming** — plain CSS custom properties; 9 switchable color themes, each with independent light/dark variants, plus a separate “glass finish” visual toggle.
- **Client-side persistence** — localStorage only (auth token, theme/language/preferences, several local-only conveniences enumerated in Section 4.11) — no IndexedDB, no service worker on the portal side.

2.3 Chrome Extension (“Halo Agent”)

- **Manifest version** — Manifest V3, module-type background service worker (not a persistent background page).
- **Key Chrome APIs** — chrome.enterprise.deviceAttributes, chrome.idle, chrome.alarms, chrome.storage.local, chrome.offscreen (required because MV3 service workers cannot call navigator.geolocation directly).
- **Real-time client** — a raw browser WebSocket connecting to the backend's gorilla/websocket endpoint.
- **Build/environment system** — a small Node.js build script (extension/build.js) reads named environments from extension/flavours.json (local, dev, qa, uat, prod, demo, multiprod, ps1, ps2) and regenerates extension/config.js per target before packaging.
- **Third-party API** — Nominatim (OpenStreetMap) reverse-geocoding, used to turn a raw GPS/Wi-Fi fix into a human-readable address. This is the only non-Google external API called by the extension.

2.4 Summary Table

Layer	Core Technology	Notes
Backend language	Go 1.22	No framework; stdlib net/http
Google integration	google.golang.org/api v0.183.0	8 API surfaces, service account + domain-wide delegation
Real-time	gorilla/websocket v1.5.3	This project's own infra, not a Google API
Database	MongoDB (mongo-driver/v2)	Optional — app runs in-memory without it
Frontend	React 18 + Babel Standalone (CDN, no build step)	Single HTML file, ~7,300 lines
Mapping	Leaflet 1.9.4	CDN
Extension	Chrome Manifest V3	Service worker + offscreen document
Geocoding	Nominatim (OpenStreetMap)	Only non-Google external API used

3. Architecture Overview

Three independent pieces make up the system, connected only through well-defined HTTP/WebSocket boundaries — there is no shared code or shared process between them.

- **The Go backend is the only component that talks to Google** — it holds the service-account credential, performs domain-wide-delegation impersonation, and exposes a REST-style JSON API under `/api/chromeos/*` and `/api/auth/*`.
- **The frontend is a pure client of that backend API** — it never talks to Google directly, and carries no Google credentials of its own.
- **The Chrome extension talks only to the backend's own endpoints (`/location`, `/idle-state`, `/ws`)** — it never touches any Google API and carries no Google credentials either; its only external dependency besides the backend is the free Nominatim geocoding service.

Two operating modes exist side by side, switched by a single environment variable (`GOOGLE_MODE=real`): a fully self-contained mock mode (a hand-built, in-memory fake Google client that fabricates a plausible small fleet with simulated network latency, for zero-config local development/demos) and real mode (the genuine, code-complete Google API integration). Both modes implement the exact same Go interface, so the rest of the backend and 100% of the frontend code is identical either way — nothing about the UI or business logic changes based on which mode is active.

4. End-to-End Feature Breakdown

This section catalogs every feature in the product, organized the way an admin encounters them: signing in, connecting the portal to Google, and then every tab in the main dashboard. Every device action is listed explicitly and in full, as requested.

4.1 Authentication & Roles

The portal has its own login system, entirely separate from Google authentication (see Section 5.5 for how the two relate). Login issues a bearer token stored in the browser, attached automatically to every subsequent API call, valid for 12 hours.

Three roles govern what a signed-in admin can do:

- **Viewer** — read-only across the entire portal. Cannot run any device action, cannot edit policies, cannot change settings.
- **Operator** — everything Viewer can do, plus: Sync now, Restart, Screenshot, Remote desktop (CRD), Capture logs, Support packet, Set volume, and the per-row “refresh this device from Google now” button.
- **Admin** — everything Operator can do, plus every destructive or administrative action: Lock/Wipe, Disable/Enable, factory Reset/Powerwash, Deprovision, Move org unit (single and bulk), Deploy Location Tracker / install any extension, Disconnect the connector, edit Sync interval and Insight thresholds, edit or clear Chrome policies, delete/undo security alerts, and edit device Asset info.

Three demo accounts ship with the portal for evaluation: admin/admin123, operator/operator123, viewer/viewer123.

4.2 Setup Wizard — Initial Connection

A three-step wizard connects the portal to a real Google Workspace domain (admin-only):

- **Step 1** — Upload service account key — the admin uploads a Google Cloud service-account JSON key file, plus the Workspace admin email to impersonate and the Customer ID. The key is parsed client-side only to confirm it looks valid before upload, and is never written to local disk on either side.
- **Step 2** — Authorize in Google Admin console — the wizard displays the exact Client ID and OAuth scope list needed, with one-click copy buttons (including “copy all scopes, comma-separated” for pasting directly into Admin console’s domain-wide delegation screen), and links straight to that Admin console page. It polls the backend until Google confirms the grant.
- **Step 3** — Connect and start syncing — finalizes the connection and starts the background sync worker, which performs the first full device/user/org-unit sync.

This reflects Google’s real, actual service-account + domain-wide-delegation authorization model end to end — it is not a cosmetic wizard.

4.3 Dashboard — Connector Status

A persistent status bar sits above every tab, showing: the connected Google account, live device count, last sync time, connection status badge, the current auto-sync interval, a “Sync now” button (rate-limited server-side so it cannot be spammed), and a “Disconnect” button (admin only).

- **Connection details panel** — an expandable detail view showing Client ID, Customer ID, connected account, authorization timestamp, and the full list of authorized OAuth scopes.
- **Notification bell** — shows an unread count of recent device events, clearing when the Activity tab is viewed.
- On the Halofort theme specifically, a proper left-sidebar “Dashboard” landing page exists, which is the Fleet Insights panel (Section 4.5) relabeled and restyled as a dashboard home.

4.4 Devices

4.4.1 Devices Table & Filters

A device-type filter (All devices / ChromeOS / Mobile) sits above two independent tables/panels. The ChromeOS devices table refreshes every 3 seconds and shows:

- **Device** — serial/name (clickable, opens the full device-detail view).
- **User** — the assigned Workspace user.
- **Org unit** — the device's current organizational unit path.
- **Status** — Google's real provisioning state (Active / Disabled / Deprovisioned / etc.).
- **Sync status** — a derived Active/Stale badge based on the configurable stale-after threshold.
- **Google sync** — an Online/Offline indicator and “time since last sync,” computed purely from Directory API data against a configurable online-after threshold. Explicitly not a live connectivity check. Includes a per-row manual refresh button that pulls that one device fresh from Google immediately.
- **Live check-in** — a genuinely real-time signal from the Location Tracker extension: either a live WebSocket connection (updates within seconds of the device actually connecting/disconnecting) or, as a fallback, the extension's last periodic location ping.
- **Actions / Location** — either inline action buttons or a link into the device's Location tab, depending on theme.

4.4.2 Bulk Actions

Admins can select multiple devices (checkboxes, including “select all”) and move all of them to a new org unit in a single real, batched Google API call — not one call per device.

4.4.3 Per-Device Actions — Complete List

Every device action available in the product, in full — each is a real, live write against the customer's Google Workspace domain (except where noted), gated by the role shown, and each destructive action requires an explicit confirmation step in the UI:

- **Restart (Operator+)** — reboots the device immediately.
- **Screenshot (Operator+)** — captures the device's current screen (kiosk/managed-guest-session devices only).

- **Remote desktop / CRD (Operator+)** — starts a Chrome Remote Desktop session and returns a real, usable session link.
- **Capture logs (Operator+)** — requests a diagnostic log capture from the device.
- **Fetch support packet (Operator+)** — requests a full device support bundle.
- **Set volume (Operator+)** — sets the device's system volume (0–100), kiosk/managed-guest-session devices only.
- **Lock / Wipe (Admin)** — forces re-login and clears the local user's data. This is explicitly not a full factory reset — labelled “Lock” in the UI to avoid that confusion.
- **Disable (Admin)** — offers a real two-choice dialog matching Google Admin console's own: “Disable With Lock Screen” (default) or “Disable With Factory Reset” (which sequences Disable followed by a Powerwash).
- **Enable / Re-enable (Admin)** — restores a disabled device to active management.
- **Move org unit (Admin)** — moves a single device to a chosen org unit (autocompletes against the real OU list).
- **Deploy Location Tracker (Admin)** — force-installs the Location Tracker Chrome extension onto every device in the device's org unit via a real Chrome Policy API write.
- **Reset / Factory reset (Admin)** — a real two-choice dialog: “Clear User Profiles” (recommended) or full “Factory Reset” — gated behind an explicit “I understand this cannot be undone” checkbox.
- **Deprovision (Admin)** — permanently removes the device from active management, with a real choice to bundle a factory reset first (default, matching Admin console's own default) or preserve data — correctly sequenced so the reset command runs before the device record is removed.
- **Per-row Google refresh (Operator+)** — re-fetches just that one device from Google immediately, without waiting for the next scheduled sync.

Not offered, by design: “Reload policies” and “Locate” actions some MDM products advertise — no public Google API exposes either of these for this integration, and the product deliberately does not fabricate an action it cannot actually perform.

4.4.4 Device Detail — Nine Tabs

- **Hardware** — device summary (model, TPM, manufacture date, installed apps preview), live CPU utilization/temperature/architecture, connected displays/audio devices/USB peripherals, and boot performance (last boot/shutdown duration and reason).
- **Battery** — charge %, status, cycle count, a computed battery-health percentage (matching Admin console's own calculation) alongside Google's coarser health enum, plus manufacturer, serial, discharge rate, temperature, voltage, and full design/rated capacity.
- **OS & updates** — Device ID/Serial (maskable), provisioning status, OS version with a Compliant/Expired badge against the Auto-Update Expiration date, boot mode, firmware version, ChromeOS vs. ChromeOS Flex, license type, pending-update state, and extended-support eligibility.
- **Network** — last known IP, Wi-Fi/Ethernet MAC, connection state (Online / Connected-no-internet / Captive portal / Connecting / Not connected), latency, signal strength, link quality, and encryption status.

- **Apps** — a paginated table of per-device app usage (name, time used, type), resolved against the fleet-wide installed-apps report for real display names.
- **Storage & Memory** — used/total storage bar with disk model/serial, and used/total memory bar with memory-encryption state.
- **Actions** — hosts every device action listed in Section 4.4.3.
- **Location** — an embedded live map with the device's last 50 real GPS/Wi-Fi pings and a movement trail, a configurable ping interval, a full location-history table, a connectivity-history table (real WebSocket connect/disconnect and idle/active/locked events), and a second independent (currently feature-flagged off) IP-based approximate location signal.
- **Usage & enrollment** — last seen/synced, today's active-usage minutes, recent user, enrollment dates, and an editable Asset info block (Location/Notes/Order #) that writes real values back to Google.

4.4.5 Mobile Devices

A separate, read-only panel listing phones/tablets under Workspace MDM (owner, model, OS, type, status, last sync) — lazy-loaded on first expand, kept entirely separate from ChromeOS device management since mobile devices carry a different data shape.

4.5 Fleet Insights & Reports

4.5.1 Fleet Insights

Organized into Overview, Directory, Distributions, Org units, Warnings & security, and Device events:

- **Overview** — headline stat tiles (total devices, stale count, average fleet age, devices over 3 years old, fleet usage minutes) plus status-distribution donuts for device status, live extension status, network visibility, provisioning status, boot mode, and update status.
- **Directory** — user/group health stat tiles and an enrollment trend chart.
- **Distributions** — breakdowns by OS version, device model, firmware version, license type, and building/location, plus a three-tier “top consumed apps” view.
- **Org units** — a table of every org unit with total/active/offline device counts.
- **Warnings & security** — fourteen distinct, real warning cards, each linking straight into a filtered Devices view: Auto-update expiration, Pending updates, Developer mode, Extended support needed, Low disk space, Low memory, Orphaned devices (assigned user is suspended), Login/sync mismatch, Shared devices, Hot CPUs, Outdated TPM, Fleet churn, Idle today, and Tracked/untracked location.
- **Device events** — a fleet-wide event stream (network, USB, app-install, audio events) with a selectable time window, an events-per-day chart, a by-type breakdown, and a recently-installed-apps list.

4.5.2 Reports

- **Chrome Versions Report, Devices Report, Chrome Insights Report** — summary cards linking into the relevant detail view, each exportable to CSV.
- **Printers Report and Chrome Crash Report** — real Chrome Management reporting data.
- **App insights** — top apps/extensions by device, and top Android apps by user.

- **App usage by device** — an on-demand, per-device app-usage pivot (not backed by any single fleet-wide Google API — built by fetching every device's own telemetry).
- **Legacy Technology Report** — permanently unavailable: no public Google API exists for it, and the product does not fabricate this data.
- **“Export all (CSV)”** — downloads every report at once, entirely client-side, no extra backend round-trip.

4.6 Security

- **Alerts** — real Google Alert Center data (including DLP). Admins can delete an alert (with a real Undo), and submit “was this useful” feedback — both genuine, live actions on Google's side.
- **Audit log** — real Admin SDK Reports API data, split into Admin activity and Login activity — a separate feed from the portal's own device-action history.
- **Policy compliance** — live-resolved values for a curated set of Chrome policies, editable (with confirmation) via the real Chrome Policy API. Includes a dedicated “Install a Chrome extension” panel (paste any Chrome Web Store extension ID and it force-installs to a chosen org unit) and a friendly add/remove editor for force-installed PWAs and Android/ARC apps.
- **Browse policies** — live keyword search against Google's real Chrome Policy schema catalog — the same mechanism app/network/kiosk/printer management are all reached through, rather than separate hardcoded integrations.
- **Admin roles** — shows who holds which real Google Workspace admin role, explicitly distinct from this portal's own login roles.

4.7 Enrollment

A guidance-only section — no public Google API exists for either enrollment path, so nothing here writes to Google. Covers Manual enrollment (step-by-step OOB instructions), Zero-touch enrollment (clearly separating the self-service steps an admin can do from the reseller-only steps no third-party app can perform), and ChromeOS Flex (converting existing hardware, positioned as a pre-step, not a third enrollment path).

4.8 Settings

- **Appearance** — theme template (9 options, Section 4.10), light/dark/system mode, glass-finish toggle.
- **Fleet insights preferences** — default sub-tab on load, optional auto-cycling through sub-tabs with a configurable interval.
- **Auto-sync (Admin)** — the sync interval, from “always (~30s)” up to hourly, or a custom value — enforced against a real 10-second server-side floor.
- **Insight thresholds (Admin)** — the numeric thresholds driving the Warnings & security cards and the Devices table's Stale/Online badges: stale-after days, online-after minutes, low disk %, low memory %, hot-CPU temperature, and outdated-TPM family.

4.9 Directory & Activity

- **Directory** — three real, read-only tables: Users, Org units (the full Workspace OU tree, not just OUs with devices), and Groups.

- **Activity** — “Recent device actions” (a session-only history of actions issued through this portal) plus the real Chrome Management telemetry event stream.

4.10 Theme Templates

Nine selectable color themes (Warm Neutral/default, Black & White, Blue & White, Slate, Forest, Minimal Gray, Ocean, Halofort, Sand), each independent of a separate light/dark/system mode toggle. Eight of the nine only change color values. The ninth — Halofort — additionally changes the whole app shell: a fixed collapsible sidebar, a full-page device-detail view instead of an inline expander, Fleet Insights relabeled as “Dashboard” and regrouped into 3 tabs instead of 6, and a category-tile layout for Policy compliance.

4.11 Real vs. Local-only Actions — Summary

For clarity on what actually changes Google Workspace state versus what is purely a local UI convenience:

- **Real, Google-backed writes** — every device action in 4.4.3; location ping-interval changes; device Asset info edits; Chrome policy set/clear (including extension/PWA/ARC installs); security alert delete/undo/feedback; sync-interval and insight-threshold changes; sync now / disconnect; the Setup Wizard's connect steps.
- **Local-only (browser storage, no Google-side effect)** — theme/mode/language selection, insights auto-cycle preferences, tab-position memory, the session-only “recent device actions” list, the “installed this session” extension-install list, custom Browse-policies schema bookmarks, and cached IP-location lookups.
- **Read-only integrations (no write path offered at all)** — Mobile devices, Audit log, Admin roles, Directory Users/Groups/Org-units, Chrome Reports, Device Events/Telemetry, Enrollment guidance, and the Legacy Technology Report (permanently locked, no API exists).

5. Google APIs & Metrics Mapping

Every metric, table, and write action in the product is traceable to one specific Google API method (or, in a small number of clearly-marked cases, computed locally from already-synced data, or sourced from a non-Google third-party API). This section is the authoritative mapping.

5.1 Google API Surfaces & OAuth Scopes

The service account requests these scopes under domain-wide delegation, split into separate token requests per group so an un-authorized new scope can never break already-working calls that share nothing with it:

#	API / Product	OAuth Scope(s)	Powers
1	Admin SDK Directory API (core)	admin.directory.device.chromeos, admin.directory.user.readonly, admin.directory.orgunit.readonly	Devices table, fleet inventory, Directory tab (Users/Org units), churn/retired-device tracking
2	Admin SDK Directory API (extended)	admin.directory.group.readonly, admin.directory.device.mobile.readonly	Groups tab, Mobile devices panel
3	Admin SDK Directory API (roles)	admin.directory.rolemanagement.readonly	Admin roles panel
4	Chrome Management API — Reports	chrome.management.reports.readonly	Chrome version breakdown, top apps/extensions, Android apps, printer usage, crash events
5	Chrome Management API — Telemetry	chrome.management.telemetry.readonly	Device detail hardware/battery/network/storage panels, per-sync connection state, device Events feed
6	Chrome Policy API	chrome.management.policy (full read+write)	Policy compliance, Browse policies, all policy set/clear writes including app installs
7	Admin SDK Reports API	admin.reports.audit.readonly, admin.reports.usage.readonly	Audit log (admin activity + login activity)
8	Admin SDK Alert Center API	apps.alerts	Security alerts, delete/undo/feedback actions

Confirmed not used: the Chrome Web Store API, any Cloud Identity API, and any user-facing “Sign in with Google” OAuth flow.

One feature uses a non-Google third-party API: approximate device location by public IP address calls ip-api.com (a free IP-geolocation service) as an enrichment/fallback on top of Directory API data — this feature is currently hidden behind a feature flag and off by default.

5.2 Device Actions → Exact Google API Method

Device Action	Google API Method Called
Restart	Chromeosdevices.IssueCommand — REBOOT
Lock / Wipe	Chromeosdevices.IssueCommand — WIPE_USERS
Reset / Factory reset (Powerwash)	Chromeosdevices.IssueCommand — REMOTE_POWERWASH
Screenshot	Chromeosdevices.IssueCommand — TAKE_A_SCREENSHOT
Set volume	Chromeosdevices.IssueCommand — SET_VOLUME

Remote desktop (CRD)	Chromeosdevices.IssueCommand — DEVICE_START_CRD_SESSION, then Commands.Get polled for the session URL
Capture logs	Chromeosdevices.IssueCommand — CAPTURE_LOGS
Fetch support packet	Chromeosdevices.IssueCommand — FETCH_SUPPORT_PACKET
Disable	Devices.BatchChangeStatus — CHANGE_CHROME_OS_DEVICE_STATUS_ACTION_DISABLE
Enable / Re-enable	Devices.BatchChangeStatus — CHANGE_CHROME_OS_DEVICE_STATUS_ACTION_REENABLE
Deprovision	Devices.BatchChangeStatus — CHANGE_CHROME_OS_DEVICE_STATUS_ACTION_DEPROVISION
Move org unit (single device)	Chromeosdevices.MoveDevicesToOu
Move org unit (bulk)	Chromeosdevices.MoveDevicesToOu (full ID list in one request)
Asset info edit (Location/Notes/Order #)	Chromeosdevices.Patch (AnnotatedLocation/Notes/OrderNumber only)
Deploy Location Tracker / Install any extension	Policies.Orgunits.BatchModify — schema chrome.users.apps.InstallType, appInstallType: FORCED
Force-install PWA / Android app (AppListEditor)	Policies.Orgunits.BatchModify — webAppInstallForceList / arcPolicy schemas

Two deliberate corrections worth recording: device status changes use the modern BatchChangeStatus endpoint rather than Chromeosdevices.Patch(Status:...), since Patch was confirmed to return 200 OK while silently not changing anything server-side. Org-unit moves similarly use MoveDevicesToOu rather than Patch(OrgUnitPath:...), since Patch was confirmed to reject the request with “Inconsistent Orgunit id and path.”

5.3 Fleet Insights Metrics — Data Source

Metric / Card	Source
Device status (Online/Offline)	Directory API last-sync vs. configurable online-after threshold
Live extension status	This app's own WebSocket/location-ping signal (not a Google API)
Provisioning status, boot mode, update status	Directory API device fields
OS version / model / firmware / license distributions	Directory API device fields, aggregated locally
Location (by building)	Directory API AnnotatedLocation field (explicitly not GPS-based)
Top consumed apps	Chrome Management Reports (fleet-wide), falling back to per-device Telemetry, then real APP_INSTALLED events
Auto-update expiration / pending updates	Directory API device fields
Developer mode / outdated TPM / hot CPUs	Chrome Management Telemetry (per-device)
Orphaned devices	Directory API device + user records, cross-referenced locally
Fleet churn (deprovisioned)	Directory API device list filtered by status, with Google's own recorded reason
Device events (network/USB/app-install/audio)	Chrome Management Telemetry — Events.List
Tracked location / live map	This app's own Chrome extension location pings (not a Google API)

5.4 Backend Endpoints — Full Reference

Legend: G = calls a real Google API; L = local/cached computation, no live Google call; N = calls a non-Google third-party API.

Endpoint	Type	Role	What it does
POST /api/auth/login	L	—	Portal login (bcrypt), issues session token
POST /api/auth/logout	L	Any	Ends session
GET /api/auth/me	L	Any	Returns current session identity/role
POST /connect/init	L	Admin	Prepares a new connection from env vars
POST /connect/upload	G	Admin	Builds a real Google client from an uploaded service-account key
GET /connect/status	G	Admin	Polls Probe() to detect domain-wide delegation authorization
POST /connect/finish	G	Admin	Finalizes connection, triggers first sync
POST /disconnect	L	Admin	Clears connection and swaps in a client that rejects all further calls
GET /status	L	Viewer+	Cached connection status
GET /devices	L	Viewer+	Cached device list (populated by sync)
GET /insights	L	Viewer+	Local aggregation over cached devices/users/org units
GET /directory	L	Viewer+	Cached users/org units/groups
GET /chrome-reports	G	Viewer+	Live Chrome Management Reports
GET /mobile-devices	G	Viewer+	Live Mobile devices list
GET /audit-log	G	Viewer+	Live Admin + Login activity
GET /security-alerts	G	Viewer+	Live Alert Center alerts
POST /security-alerts/action	G	Admin	Delete / undo / feedback on an alert
GET /policies	G	Viewer+	Live-resolved curated policy values
GET /policies/search	G	Viewer+	Live schema catalog keyword search
GET /policies/resolve	G	Viewer+	Live-resolve one specific schema
POST /policies/set	G	Admin	Real, enforced policy write
POST /policies/clear	G	Admin	Reverts a policy to inherited default
GET /admin-roles	G	Viewer+	Live Workspace admin role assignments
GET /devices/telemetry	G	Viewer+	Live per-device hardware/battery/network telemetry
GET /devices/ip-location	N	Viewer+	Non-Google IP geolocation lookup (feature-flagged off)
GET /device-events	G	Viewer+	Live per-device or fleet-wide telemetry events
POST /location	L	Extension (unauth.)	Chrome extension's location ping ingestion
POST /idle-state	L	Extension (unauth.)	Chrome extension's idle/active/locked report
GET /ws	L	Extension (unauth.)	WebSocket upgrade for live check-in
POST /location/interval	L	Admin	Sets the extension's ping interval
POST /sync	G	Operator+	Full re-sync (devices, users, org units, churn, groups)
POST /devices/refresh	G	Operator+	Single-device live refresh
POST /devices/action	G	Operator/Admin (per action)	Dispatches any device action (Section 5.2)
POST /devices/batch-move	G	Admin	Bulk org-unit move
POST /devices/asset-info	G	Operator+	Writes AnnotatedLocation/Notes/OrderNumber

GET/POST /sync/settings	L	Read any, write Admin	Sync interval
GET/POST /thresholds	L	Read any, write Admin	Insight thresholds

5.5 Authentication — Two Separate Systems

It is worth stating explicitly: portal login (Section 4.1) and Google API authentication are completely independent systems that never overlap.

- **Portal login** — this app's own bcrypt-hashed accounts and bearer-token sessions, governing what an admin can click inside this portal.
- **Google API authentication** — a service-account JSON key, used to build a JWT and impersonate a specific Workspace super-admin/delegated-admin via domain-wide delegation, for every Directory/Chrome Management/Policy/Reports/Alert Center call. A portal Admin can upload or replace this key, but signing into the portal itself never touches Google.

5.6 Mock vs. Real Mode

A single switch point (an environment variable, `GOOGLE_MODE=real`) decides whether the backend talks to a fully self-contained, hand-built in-memory fake Google client (a small, plausible fixed fleet with simulated network latency, for zero-config local development) or the genuine, code-complete real client built on Google's own official client libraries. Both implement the exact same internal interface, so no other backend code — and none of the frontend — differs between the two modes.

5.7 Data Persistence

MongoDB is entirely optional: if it isn't reachable at startup, the app logs a warning and continues running fully in-memory (nothing survives a restart in that case). When available, it persists the connection record, device list, action history, sync/threshold settings, cached directory data, portal login sessions, and portal user accounts. One thing is deliberately never persisted anywhere, on disk or in the database: the Google service-account credential itself. A restart always requires either environment-variable credentials or a fresh upload through the Setup Wizard — this is an intentional security choice, not an oversight.

6. Chrome Extension — How It Works and Usage

The companion Chrome extension, “Halo Agent” (Manifest V3), is a lightweight, entirely invisible background agent installed on managed ChromeOS devices. It has no popup, no browser-action icon interaction, and no content scripts — the device's end user sees nothing at all in normal use. Its only purpose is reporting three signals Google's own APIs cannot provide in real time: device location, live connectivity, and idle/active/locked state.

6.1 Permissions and Why Each Exists

Permission	Purpose
<code>enterprise.deviceAttributes</code>	Resolves the device's real Google Admin/Directory device ID
<code>offline</code>	Lets the MV3 service worker run a geolocation call (which it cannot do directly)
<code>alarms</code>	Schedules the periodic location ping and a WebSocket-reconnect safety net
<code>idle</code>	Detects active/idle/locked transitions
<code>geolocation</code>	Backs the offline document's actual position request
<code>storage</code>	Diagnostic log, offline ping queue, device ID cache, manual ID override

6.2 Device Identification

The extension always tries `chrome.enterprise.deviceAttributes.getDirectoryDeviceId()` first to get the device's real Directory ID. Chrome only actually returns a value from this call for extensions that are force-installed via enterprise policy — a sideloaded (“Load unpacked”) install used for local testing will have the permission declared but Chrome's platform check will silently return empty. For that case, a manual device-ID override (set once per test device via the extension's options page) is used as a fallback. If neither resolves, the extension logs the failure and does not publish under a wrong or placeholder identity.

6.3 Location Tracking

A location ping can be triggered by any of: a recurring alarm (default 15 minutes, server-adjustable), device/browser startup, an idle-to-active transition, or a manual “Force Sync Now” button on the options page.

Each ping: flushes any points queued from a previous offline stretch first, resolves the device ID, requests a GPS/Wi-Fi position via the offline document (30-second timeout, since ChromeOS has no GPS chip and Wi-Fi-based positioning routinely takes longer indoors), best-effort reverse-geocodes the coordinates into a human-readable address, and queues the result (capped at 50 points, oldest dropped first) before attempting to publish it immediately.

6.4 Real-Time “Live Check-in” (WebSocket)

A persistent WebSocket connection gives the backend a genuinely real-time online/offline signal — within seconds of an actual disconnect, not a polling guess. It includes a 20-second application-level heartbeat on top of the transport's own ping/pong, and a staleness check (90 seconds without a confirmed round trip) that force-reconnects even if the local socket handle still incorrectly claims to

be open — a scenario confirmed to happen across a Manifest V3 service-worker suspend/resume cycle. Reconnection uses exponential backoff, from 2 seconds up to a 60-second cap.

6.5 Idle / Activity State

Every transition between active, idle, and locked is reported to the backend — this is the only way the server can ever know about idle or locked state at all, since nothing server-side can observe it independently. A transition back to active also triggers an immediate location ping and a WebSocket reconnect attempt.

6.6 Diagnostics and Troubleshooting

A rolling, capped diagnostic log (200 entries) is written for every meaningful event — lifecycle, device-ID resolution, geolocation, publishing, authentication, WebSocket, and idle events — and is viewable from the extension's own options page (no DevTools or Developer Mode required, since the “inspect service worker” DevTools link is not reliably reachable on real managed devices).

To troubleshoot a device from its options page:

- **Check whether a device ID has resolved at all** — if not, the enterprise API isn't granting one (likely a sideloaded test install) and a manual override hasn't been set.
- Scan the log for error-highlighted rows to see exactly which stage is failing (geolocation, publish, auth, or WebSocket).
- **Check the pending-queue count** — a persistent backlog implies a network or backend issue rather than a one-off blip.
- Use “Force Sync Now” to reproduce an issue on demand and watch new log rows appear live.

6.7 Manual Device ID Override

Set once per device from the extension's options page (an input field plus Save button). This exists specifically so multiple sideloaded test devices don't all silently fall back to reporting under the same placeholder ID and overwrite each other's location records — each test device gets its own explicit override. Leaving the field blank and saving clears the override, returning to relying on the enterprise API alone.

6.8 Deployment

- **Sideload (development/testing)** — “Load unpacked” via `chrome://extensions` in Developer Mode. Requires setting a manual device ID override per test device (Section 6.7), since the enterprise API will not resolve one.
- **Force-install by policy (production)** — the standard, supported path: the extension is pushed via Chrome device/user policy (“Force-installed apps and extensions” in Admin console), targeted at an org unit. This is required for device-ID resolution to work as designed.
- **Environment/build system** — a small Node.js build script regenerates the extension's configuration (API URLs, auth endpoints, tenant ID) per target environment (local, dev, qa, uat, prod, demo, and others) before packaging, so the same source ships correctly to each environment.

6.9 End-User Impact

Entirely invisible in normal operation: no popup, no notifications, no on-screen indication that location or activity state is being reported. The only surface an end user could ever encounter is Chrome's native geolocation permission prompt, and on a properly force-installed managed device this is typically pre-granted by policy rather than shown at all.

7. Development Approach — Manual vs. AI-Assisted

This section is an honest estimate, not a precise scientific measurement — it is presented that way deliberately, since anything more exact would be overstating what can actually be known.

7.1 What Actually Happened (AI-Assisted)

This project was built through iterative, conversational AI-assisted development — the working codebase spans roughly three calendar weeks (git history from August 19, 2026 to September 8, 2026), built up across many focused sessions of interactive development rather than in one continuous push. Each session typically covered one feature, one bug investigation, or one integration end to end: specify the goal, implement it, verify it live (often via automated browser-driven testing), and iterate immediately on anything found wrong — all within the same conversation.

Metric	Value
Calendar span (first to latest commit)	~3 weeks (Aug 19 – Sep 8, 2026)
Backend code (Go)	~6,700 lines across 4 files
Frontend code (single HTML file)	~7,300 lines
Chrome extension code	~1,000 lines
Distinct Google API surfaces integrated	8, plus 1 non-Google (Nominatim/ip-api)
UI theme variants shipped	9 color themes × light/dark, plus one alternate app shell

7.2 Manual-Equivalent Estimate (Industry Baseline)

For comparison, here is a standard industry-rule-of-thumb estimate for a small engineering team building the equivalent scope from scratch by hand: a real, working integration with eight separate Google Workspace/Chrome Enterprise API surfaces (each requiring its own scope research, authentication testing, and error-handling discovery against Google's actual live behavior), a themable single-page admin UI covering every feature in Section 4, a Manifest V3 Chrome extension correctly handling service-worker lifecycle quirks (offscreen documents, alarm/idle wake triggers, WebSocket staleness detection), and a real-time backend layer with its own reconnect/race-condition handling.

Work Area	Manual Estimate (2-3 engineer team)
Google API integration & auth (8 surfaces)	3–5 weeks — scope discovery, DWD auth, error handling per API
Backend device/action/policy engine	3–4 weeks
Real-time WebSocket layer + reconnect/race handling	1.5–2 weeks
Frontend — full feature set (Section 4)	5–7 weeks
Theming (9 variants) & alternate app shell	1.5–2 weeks
Chrome extension (MV3 quirks, offline queue, diagnostics)	2–3 weeks
Integration testing, bug-fixing, polish	2–3 weeks
Total (parallelized across a 2–3 person team)	3–4.5 months elapsed

The comparison, in short: AI-assisted iterative development compressed roughly 3–4.5 months of typical small-team manual engineering effort into about 3 weeks of calendar time, with each feature verified working (often via live, automated browser testing) before moving to the next — the

difference comes primarily from eliminating the research/trial-and-error time normally spent discovering each Google API's real, undocumented behavior (exact error codes, which endpoint actually works vs. silently no-ops, correct request shapes), rather than from skipping any of that verification work itself.

